

Lista de Exercícios — Cálculo Numérico

Propagação de Erros, Método de Newton-Raphson e Método da Secante

Prof. Paulo César Linhares da Silva
Universidade Federal Rural do Semi-Árido (UFERSA)

Instruções: Resolva as questões apresentando todo o desenvolvimento teórico utilizado. Nas questões que exigem implementação computacional, apresente o código-fonte (em Python ou linguagem de sua preferência), a saída obtida e a interpretação dos resultados.

Questão 1 — Propagação de Erros na Potenciação

Em sistemas de processamento digital de sinais, a potência de um sinal elétrico medido em um circuito é frequentemente estimada a partir da amplitude da tensão V , através da relação

$$P = V^n$$

em que n é um expoente característico do modelo de carga utilizado. Suponha que, em um experimento de instrumentação para um sistema embarcado, a tensão medida seja

$$V = 12,40 \text{ V}, \quad \Delta V = 0,05 \text{ V}, \quad n = 3$$

sendo ΔV o erro absoluto de medição do voltímetro digital utilizado.

- (a) Enuncie o **Teorema Geral do Erro** (erro propagado por diferenciação) e aplique-o para deduzir a expressão do erro absoluto propagado ΔP para uma função do tipo $P = V^n$.
- (b) Calcule o valor numérico de P e do erro absoluto propagado ΔP .
- (c) Determine o erro relativo percentual de P e discuta se a precisão obtida é adequada para um sistema embarcado de monitoramento de energia que exige erro relativo inferior a 2%.

Questão 2 — Propagação de Erros na Radiciação

Em algoritmos de *Ciência da Computação* que avaliam a complexidade de certas estruturas de dados (por exemplo, o tempo médio de busca em árvores balanceadas), é comum que uma métrica de desempenho T dependa da raiz de um parâmetro N relacionado ao número de elementos processados, segundo o modelo

$$T = \sqrt[3]{N}$$

Em um teste de desempenho realizado em um cluster computacional, o número de elementos processados foi estimado, por meio de contadores de desempenho (*performance counters*), como

$$N = 8000, \quad \Delta N = 40$$

em que ΔN representa a incerteza de contagem decorrente de concorrência entre processos (*race conditions* de medição).

- (a) A partir do Teorema Geral do Erro, deduza a expressão geral do erro absoluto propagado ΔT para uma função de radiciação do tipo $T = \sqrt[n]{N} = N^{1/n}$.
- (b) Calcule T e o erro absoluto propagado ΔT para os dados fornecidos.
- (c) Calcule o erro relativo de T e comente o impacto desse erro na confiabilidade da métrica de desempenho utilizada para comparar dois algoritmos concorrentes.

Questão 3 — Método de Newton-Raphson (Resolução Computacional)

Em problemas de *aprendizado de máquina*, uma das etapas de treinamento de certos modelos consiste em encontrar o ponto em que a derivada de uma função de custo se anula ou, de forma equivalente, encontrar a raiz de uma função associada ao gradiente do modelo.

Considere que, em um problema simplificado de otimização de um sistema de alocação de recursos em um *data center*, o consumo energético líquido do sistema, em função da carga de processamento x (em unidades normalizadas), seja modelado pela função

$$f(x) = x^3 - 2x - 5$$

Deseja-se encontrar o valor de carga x para o qual o sistema atinge o ponto de equilíbrio energético, ou seja, a raiz real de $f(x)$.

- (a) Enuncie o método de Newton-Raphson, apresentando sua fórmula de recorrência e as condições de convergência.
- (b) **Implemente um programa** (em Python, ou outra linguagem de sua preferência) que aplique o método de Newton-Raphson para encontrar a raiz de $f(x)$, utilizando como aproximação inicial $x_0 = 2$ e critério de parada $|f(x_k)| < 10^{-6}$ ou número máximo de 20 iterações.
- (c) Seu programa deve exibir, para cada iteração: o valor de x_k , $f(x_k)$, $f'(x_k)$ e o erro relativo aproximado.
- (d) Apresente a tabela de resultados obtida, o valor final da raiz encontrada e o número de iterações necessárias para a convergência.

Sugestão de esqueleto de código (Python):

```
1 def f(x):
2     return x**3 - 2*x - 5
3
4 def df(x):
5     return 3*x**2 - 2
6
7 def newton_raphson(x0, tol=1e-6, max_iter=20):
8     x = x0
9     for k in range(1, max_iter + 1):
10         fx = f(x)
11         dfx = df(x)
12         if dfx == 0:
13             raise ZeroDivisionError("Derivada nula durante a iteracao.")
14         x_novo = x - fx / dfx
15         erro = abs((x_novo - x) / x_novo)
16         print(f"Iteracao {k}: x = {x_novo:.8f}, f(x) = {f(x_novo):.8f},
17         erro = {erro:.8e}")
18         if abs(f(x_novo)) < tol:
19             return x_novo, k
20         x = x_novo
21     return x, max_iter
22
23 raiz, n_iter = newton_raphson(2.0)
24 print(f"\nRaiz aproximada: {raiz:.8f} obtida em {n_iter} iteracoes.")
```

Questão 4 — Método da Secante

Em *redes de computadores*, o dimensionamento de um sistema de filas para roteadores que atendem tráfego de pacotes pode ser modelado por uma função não linear que relaciona a taxa de utilização do enlace x (com $0 < x < 1$) ao tempo médio de espera dos pacotes na fila. Um modelo simplificado, obtido empiricamente para uma rede corporativa, é dado por

$$g(x) = e^x - 4x$$

O engenheiro de redes deseja encontrar o valor de utilização x para o qual o modelo indica mudança de regime (raiz de $g(x)$ no intervalo de interesse), de modo a definir o limiar de ativação do controle de congestionamento.

- Enuncie o método da Secante, destacando suas diferenças em relação ao método de Newton-Raphson (em especial, quanto à necessidade de cálculo da derivada).
- Utilizando as aproximações iniciais $x_0 = 0,1$ e $x_1 = 1,0$, aplique o método da Secante **manualmente**, realizando ao menos 4 iterações, para aproximar a raiz de $g(x)$ associada ao limiar de congestionamento. Apresente todos os cálculos.
- Em seguida, verifique seus resultados implementando um pequeno programa que execute o método da Secante para o mesmo problema, com critério de parada $|g(x_k)| < 10^{-6}$.
- Compare os resultados obtidos manualmente e computacionalmente, discutindo a velocidade de convergência do método da Secante em relação ao método de Newton-Raphson da Questão 3.